



CONTROLLED TECHNICAL RECORD

Technical Production Runbook - Phase 1

Deployment, recovery, monitoring and incident response

Field	Controlled value
Document ID	UMWS-P1-TR-001
Version	1.0
Issue date	22 August 2026
Technical owner	Jose
Client operations owner	Raisa
Sales-pause authority	Raisa
Emergency channel	Direct Jose-Raisa WhatsApp conversation
Classification	Operational - contains no secret values

SAFETY RULE

This runbook identifies systems, commands and recovery controls but never contains passwords, tokens, private keys, recovery codes or full database connection strings. Retrieve secrets only from the approved secret store at execution time.

Runbook objectives

- Restore safe customer and administrator operation without undocumented developer knowledge.
- Prefer reversible actions and isolate recovery tests before production changes.
- Preserve evidence, release identifiers and an incident timeline.
- Separate technical response (Jose) from the sales-pause decision (Raisa).

1. Production topology and ownership

Component	Production position	Control / owner
Storefront	Vercel; apex, AO and PT hosts	Jose controls Vercel and GitHub
CMS and admin	Railway; cms.usemewithstyle.shop	Jose and Raisa control Railway; Jose responds
Database	Railway PostgreSQL 18	Jose recovery operator; Raisa can escalate
Media	Cloudflare R2 / S3-compatible storage	Jose recovery operator; shared account control
Email	Resend; support/orders sender domain	Jose technical; shared recovery and mailbox
Messaging	Meta / Instagram webhook and API	Shared account control; Jose technical
DNS	Namecheap	Jose and Raisa
Monitoring	GitHub Actions and shared operations mailbox	Jose response; Jose and Raisa monitor mailbox

Production hosts

- <https://usemewithstyle.shop> - geo-routing entry point.
- <https://ao.usemewithstyle.shop> - Angola catalogue and checkout.
- <https://pt.usemewithstyle.shop> - Portugal catalogue and checkout.
- <https://cms.usemewithstyle.shop> - Payload CMS and API.

Current release model

Phase 1 uses one focused feature branch per repository, pull-request review, verified merge to main and production verification. Vercel and Railway deploy from their connected repositories. A separate persistent staging environment is a Phase 2 delivery-control improvement; production recovery must therefore rely on verified commits, provider deployment history and isolated data restoration.

2. Release and deployment procedure

Pre-deployment gate

- Confirm scope and authorized change owner.
- Confirm focused branch and clean worktree; preserve unrelated local files.
- Run repository-specific lint, tests and production build.
- Review migration and environment-variable impact.
- Record current main commit and current healthy deployment before merge.

Platform release

- 1 Create a focused branch from current main in use-me-with-style-platform.
- 2 Implement and verify locally. Run `npm test`, `npm run lint` and `npm run build` unless the change has a stricter test plan.
- 3 Push the branch, open a pull request and review the exact diff and CI results.
- 4 Merge only after checks pass. Confirm Vercel production deployment is READY and tied to the expected main commit.
- 5 Verify apex, AO, PT, representative catalogue/product pages, admin login boundary and same-origin API reads.

CMS release

- 1 Create a focused branch from current main in use-me-with-style-cms.
- 2 Run the relevant tests, `npm run lint` and `npm run build`. Review Payload migration and PostgreSQL effects.
- 3 Push, open a pull request and merge only after verification.
- 4 Confirm Railway deployment success for the expected main commit and inspect startup/migration logs.
- 5 Verify the public CMS endpoint, admin access, representative product/category reads and dependent storefront calls.

SCHEMA WARNING

Do not roll application code backward across an incompatible database migration. Determine migration compatibility and preserve a verified backup before any schema-affecting release or rollback.

3. Post-deployment validation and rollback

Minimum validation

```
curl -fsS https://usemewithstyle.shop/ >/dev/null
curl -fsS https://ao.usemewithstyle.shop/ >/dev/null
curl -fsS https://pt.usemewithstyle.shop/ >/dev/null
curl -fsS 'https://cms.usemewithstyle.shop/api/products?limit=1' >/dev/null
```

- Confirm Vercel/Railway deployment status and expected Git commit.
- Run the production operations monitor from GitHub Actions and require a green result.
- Exercise only safe representative reads unless a controlled UAT mutation was authorized.
- Record timestamp, commit, deployment URL/ID, checks and operator.

Rollback decision

Condition	Preferred action
Frontend regression; CMS/data healthy	Promote/redeploy the last known-good Vercel deployment or revert the offending commit through a focused PR.
CMS code regression; schema compatible	Redeploy the prior known-good Railway revision or revert through a focused PR, then verify CMS and storefront reads.
Schema or data may have changed	Pause. Preserve evidence and backup. Restore into an isolated database first; do not blindly redeploy older code.
Provider outage	Do not change verified application code. Confirm provider status and use the documented fallback/operational communication.

Rollback completion

- Expected release is active.
- Production monitor passes.
- Representative customer and admin paths pass.
- Database and inventory integrity are confirmed.
- Raisa receives status and decides whether sales resume.

4. PostgreSQL backup and isolated restore

Backup control

GitHub workflow PostgreSQL encrypted backup runs daily at 02:20 UTC. It creates a PostgreSQL 18 custom-format dump, validates the restore manifest, encrypts it with AES-256-CBC/PBKDF2 (200,000 iterations), deletes plaintext and retains the encrypted artifact for 14 days. Operational targets: RPO 24 hours; RTO 30 minutes.

Restore preconditions

- Jose is the recovery operator.
- Download the exact successful artifact and record its workflow run and SHA-256.
- Retrieve the passphrase from the approved secret store; never paste it into a command history, ticket or chat.
- Provision a new isolated PostgreSQL database. Never test the restore over production.

Isolated restore skeleton

```
# Set values in the current protected shell; do not log them.
openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 \
  -in useme-postgres-RUN_ID.dump.enc -out restore.dump \
  -pass env:BACKUP_ENCRYPTION_PASSPHRASE

pg_restore --list restore.dump >/dev/null
pg_restore --no-owner --no-privileges \
  --dbname="$ISOLATED_DATABASE_URL" restore.dump
```

Validation

- Compare expected table count and representative row counts with preserved source evidence.
- Run application read checks against the isolated database only.
- Confirm no production connection string was used.
- Remove plaintext dump securely after the controlled exercise; retain only approved encrypted evidence.

PRODUCTION RECOVERY

Replacing or redirecting the production database requires a separate incident decision, a verified isolated restore, an impact assessment and Raisa's sales-pause/resume decision. Never point production at an unverified restore.

5. Cloudflare R2 media recovery

Scope

Media recovery is key-specific. The Gate 3 rehearsal proved upload, download, second-key recovery, checksum equality and exact cleanup. A recovery must never use a broad prefix deletion or overwrite an existing object without explicit validation.

Safe recovery sequence

1	Identify the exact missing or corrupted media record, expected R2 bucket/key, database reference and affected pages.
2	Perform read-only HEAD/GET checks for that exact key. Detect redirects, unexpected content type and zero-length data.
3	Locate the approved source: retained original, verified backup copy or known-good object. Calculate and record SHA-256.
4	Upload first to a unique temporary recovery key under a controlled <code>_recovery-test/</code> prefix.
5	Download the temporary object, compare SHA-256 and visually inspect the image.
6	Only if the production key is confirmed absent, upload/copy the verified object to that exact key. If a conflicting object exists, stop and escalate instead of overwriting.
7	Verify the CMS URL and every known storefront use. Delete only the exact temporary key and confirm its absence.

Evidence record

- Timestamp, operator, bucket and redacted key reference.
- Source and recovered SHA-256.
- Exact temporary key and cleanup confirmation.
- Affected CMS record and storefront validation.
- No credentials or signed URLs retained in evidence.

6. Inventory reservation cleanup and heartbeat

Execution paths

Path	Schedule	Purpose
Railway scheduled service	Every 5 minutes	Primary npm run cron:release-inventory execution.
GitHub inventory heartbeat	Nominally every 5 minutes	Independent idempotent execution and durable run evidence.
Operations monitor	Every 15 minutes	Checks platform health and heartbeat freshness.
AUTHORITATIVE THRESHOLD Alert when the latest successful independent heartbeat is more than 45 minutes old. The older 15-minute wording is superseded because GitHub scheduled runs were observed 25-35 minutes late. The monitor remains fail-closed beyond 45 minutes.		

Manual recovery command

```
PAYLOAD_PUBLIC_SERVER_URL=https://cms.usemewithstyle.shop \  
CRON_SECRET="$CRON_SECRET" npm run cron:release-inventory
```

Response procedure

- 1 Check the latest Railway scheduler and GitHub heartbeat runs; distinguish provider delay from endpoint failure.
- 2 Check CMS availability and structured cleanup failure events.
- 3 If required, execute the manual command once with the secret supplied securely.
- 4 Record released-reservation count and reconcile any affected saleable stock.
- 5 Run/confirm a fresh heartbeat and production monitor. Inform Raisa if stock integrity was uncertain.

7. Monitoring, alerting and email canary

Control	Frequency	Success condition
Production operations monitor	15 minutes	Apex/AO/PT HTML, CMS API, Meta webhook handshake and heartbeat all pass.
Inventory cleanup heartbeat	Nominal 5 minutes	Cleanup endpoint succeeds; run evidence retained.
Email delivery canary	Daily 08:15 UTC	Resend accepts canary to shared operations mailbox and message is received.
Database backup	Daily 02:20 UTC	Encrypted artifact uploaded; manifest validation passed.

Alert routing

- Primary mailbox: usemewithstyle.master@gmail.com, controlled by Jose and Raisa.
- GitHub workflow failure is the fallback signal when Resend itself is unavailable.
- Jose responds to storefront, CMS, email/webhook and missed-heartbeat incidents.
- Raisa controls the sales-pause decision and customer operations response.

Controlled monitor run

```
# GitHub UI: Actions > Production operations monitor > Run workflow
# Normal check: simulate_missed_heartbeat = false
# Alert rehearsal only: true, with owner authorization
```

DO NOT SILENCE

Do not disable a failing check merely to make the dashboard green. Identify provider delay, configuration drift or application failure, document the disposition and restore a meaningful signal.

8. Email and sender-domain incident procedure

Detection

- Daily canary missing or GitHub canary workflow failed.
- Application logs show provider rejection or template delivery error.
- Customer reports non-receipt across multiple messages.
- Resend sender-domain or DNS status is no longer verified.

Response

- 1 Treat the admin order record as authoritative; do not repeat order status transitions solely to resend email.
- 2 Check Resend service status, sender-domain verification and the production API configuration.
- 3 Check Namecheap DNS records without editing until the expected/current values are compared.
- 4 Inspect sanitized application and GitHub workflow logs; never expose recipient data or API keys.
- 5 Send a controlled canary and one authorized template test to Jose.
- 6 Require provider acceptance and real mailbox receipt before closure. Raisa coordinates any manual customer follow-up.

Closure evidence

- Canary workflow run ID and timestamp.
- Sender domain verified.
- Authorized template types received.
- No repeated financial/order mutation used as a resend mechanism.

9. Meta / Instagram webhook and token operations

Monitoring boundary

The production monitor verifies the Meta webhook handshake. Live inbound/outbound messaging requires a separately authorized controlled account test. If messaging is unavailable, treat the channel as down and use the approved operational fallback; do not claim a round trip passed without evidence.

Incident response

- 1 Check Meta platform/app status and the production webhook configuration.
- 2 Confirm the callback URL and verification token placement without exposing the token.
- 3 Inspect CMS webhook logs for signature failure, invalid payload, permission or token errors.
- 4 Check access-token validity, scopes, page/Instagram account association and expiry.
- 5 After correction, require a passing verification handshake and, when authorized, one inbound/outbound round trip.

Manual token rotation

- 1 Schedule rotation before expiry and confirm Jose and Raisa retain Meta account recovery access.
- 2 Generate/extend the token through the approved Meta procedure. Do not use the removed automatic renewal workflow.
- 3 Update the token only in the approved production secret location. Never commit it or paste it into Notion, Linear, GitHub issues or chat.
- 4 Redeploy/restart only the service that reads the secret, then verify handshake and authorized messaging.
- 5 Revoke the prior token when safe and record date, owner, expiry and test result without recording the value.

SIGNATURE RULE

Webhook signatures must remain enforced. Never bypass signature verification to restore messaging; fix configuration or provider ownership instead.

10. Secret and environment-variable rotation

Secret placement

Secret class	Approved location	Rule
Frontend public configuration	Vercel environment variables with VITE_ prefix	Browser-visible by design; never place server secrets here.
CMS/database/provider secrets	Railway/Vercel server environment or GitHub Actions secrets	Production values only in production scope.
Workflow configuration	GitHub Actions secrets/variables	Use secrets for credentials; variables only for non-secret routing values.
Backup passphrase	GitHub Actions secret and Jose macOS Keychain	Never document, log or message the value.

Rotation sequence

1	Identify owner, consumers, expiry and blast radius. Record the change window.
2	Create the new credential with minimum required permissions.
3	Update one controlled environment at a time and preserve the prior credential until verification when the provider supports overlap.
4	Redeploy/restart affected services and run the relevant health/functional checks.
5	Revoke the old credential, confirm monitoring remains green and update the no-secret access register metadata.
EMERGENCY ROTATION	If exposure is suspected, revoke first when continuing use creates greater risk, pause affected operations, rotate all dependent locations, inspect logs and preserve an incident record.

11. Incident severity, command and sales pause

Severity	Definition / examples	Required response
SEV-1 Critical	Security/data exposure; order capture, payment confirmation or inventory integrity unsafe; storefront/CMS broadly unavailable.	Immediate Jose-Raisa escalation. Raisa decides pause. Preserve evidence and restore from known-good state.
SEV-2 High	Major workflow degraded; email, invoices, CMS or messaging unavailable but authoritative records remain safe.	Jose investigates promptly; Raisa coordinates manual operations and decides whether scope requires pause.
SEV-3 Normal	Limited presentation/content defect or non-critical operational issue.	Record, prioritize and repair through normal focused-branch release.

Incident command

- Technical lead: Jose.
- Client operations lead and sole sales-pause authority: Raisa.
- Emergency channel: direct Jose-Raisa WhatsApp conversation.
- Shared evidence mailbox: usemewithstyle.master@gmail.com.

Pause-sales recommendation triggers

- Checkout or order capture cannot be trusted.
- Inventory accuracy cannot be demonstrated.
- Customer data may be exposed.
- Production cannot be monitored or safely recovered.
- CMS failure prevents safe order/catalogue operations.

Resume criteria

- Root cause contained or safe workaround approved.
- Relevant monitor and representative workflow pass.
- Data and stock integrity confirmed.
- Known-good release and evidence recorded.
- Raisa explicitly authorizes sales to resume.

12. Scenario runbooks

Scenario	Technical response	Recovery proof
Storefront outage	Check all hosts, Vercel deployment/logs and CMS dependency. Promote/redeploy known-good release when appropriate.	All storefront checks HTTP 200/HTML; monitor green; representative browse/cart path passes.
CMS/admin outage	Check Railway service/logs, DB connectivity, env configuration and latest deploy. Use isolated restore procedure if data recovery is required.	CMS API/admin/representative reads pass; DB integrity confirmed; monitor green.
Email failure	Check Resend, sender DNS, API config and logs. Keep admin order state authoritative.	Canary and authorized template received; provider acceptance recorded.
Meta webhook failure	Check app/webhook, verification, signature logs, scopes and token validity.	Handshake passes and authorized round trip passes when allowed.
Missed heartbeat	Check Railway/GitHub scheduler, run cleanup once if needed and reconcile stock.	Fresh heartbeat, successful cleanup and monitor green.
Media object loss	Identify exact key, validate source/checksum via temporary key, restore only absent exact key.	Checksum and visual validation pass; affected pages load; temporary key removed.

Provider incident rule

When Vercel, Railway, GitHub, Cloudflare, Resend or Meta reports a provider incident before application initialization or networking succeeds, preserve the verified application revision. Do not make speculative code changes. Re-run the unchanged deployment or monitor after the provider confirms recovery.

13. Evidence, communication and closure

Required incident record

- Start/end timestamps and severity.
- Reporter, technical lead, operations lead and decision owner.
- Affected market, service, customers and business capability.
- Release commit/deployment ID and last known-good reference.
- Sanitized detection evidence and timeline of actions.
- Data/stock integrity checks, recovery proof and monitor run.
- Sales pause/resume decisions by Raisa.
- Follow-up owner and target phase/date.

Communication rules

- State verified facts, impact and next action; label assumptions.
- Do not expose secrets or unnecessary personal data.
- Do not tell customers payment or fiscal integrations are active when they are deferred.
- Use the direct Jose-Raisa WhatsApp channel for emergencies and the shared mailbox for durable alert evidence.

Closure checklist

- Service restored and representative functional checks pass.
- Monitoring is green and meaningful.
- Database, inventory and media integrity confirmed where relevant.
- Temporary recovery resources cleaned up exactly.
- Old/revoked credentials removed where relevant.
- Raisa authorizes sales resume if paused.
- Evidence and follow-up actions recorded.

14. Quick reference and acceptance

Control	Current value
Storefront monitoring	Apex, AO and PT every 15 minutes
CMS monitoring	Public CMS API every 15 minutes
Inventory freshness	Fail closed when successful heartbeat is older than 45 minutes
Email canary	Daily at 08:15 UTC
Database backup	Daily at 02:20 UTC; encrypted; 14-day retention
Database targets	RPO 24 hours; operational RTO 30 minutes
Alert mailbox	usemewithstyle.master@gmail.com
Technical owner	Jose
Sales-pause authority	Raisa

Authoritative source records

- CMS docs/operations-observability.md
- CMS Gate 3 production resilience evidence (21 Aug 2026)
- CMS Gate 4.5 operational tabletop evidence (22 Aug 2026)
- Platform docs/environments.md
- Platform docs/production-domain-cutover.md
- GitHub production workflow definitions on CMS main

OWNER TEST

Jose is the runbook owner and has rehearsed the recovery controls documented by Gate 3 and Gate 4.5. Final pack approval belongs to Raisa. Training and formal sign-off remain Gate 5.7 / Gate 6 activities.